Transformada de Fourier Discreta (DFT)

Ingeniería Electrónica de Comunicaciones

Jesús Chacón Sombría

Departamento de Arquitectura de Computadores y Automática.
Universidad Complutense de Madrid

Curso 2025-2026

Motivación

- La **Transformada Rápida de Fourier (FFT)**:
  - Algoritmo eficiente para calcular la DFT.
  - Reduce la complejidad computacional de $O(N^2)$ a $O(N \cdot \log_2 N)$.
- *Aplicaciones: Procesamiento de audio, imágen, comunicaciones...*

¿Por qué es importante reducir la complejidad computacional?

FFT: Cómputo eficiente de la DFT

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \qquad x[n] = \frac{1}{N} \sum_{n=0}^{N-1} X[k] e^{j\frac{2\pi}{N}kn}$$

- Para simplificar las expresiones, definimos $W_N = e^{-j\frac{2\pi}{N}}$.
- Se tiene que $X(k) = \sum_{n=0}^{N-1} x[n] W_N^{kn}$ con k=$0, 1, \ldots, N-1$:

$$\begin{bmatrix} X[0] \\ X[1] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{bmatrix} 1 & 1 & \ldots & 1 \\ 1 & W_M^1 & \ldots & W_M^{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & W_M^{N-1} & \ldots & W_M^{[N-1(N-1)} \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \\ \vdots \\ x[N-1] \end{bmatrix}$$

- El cálculo directo no es computacionalmente eficiente: $O(N^2)$
- Existen algoritmos alternativos que reducen la complejidad a $O(N \cdot \log(N))$.
- Estos algoritmos constituyen la **FFT (Fast Fourier Transform)**.

FFT: Cómputo eficiente de la DFT

Ejemplo: Algoritmo en base 2 de diezmado temporal (hay algoritmos más rapidos)

- Suponemos que la longitud N es una potencia de 2 ($N = 2^p$).
- Dividimos la secuencia $x[n]$ en dos secuencias de longitud $N/2$:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk} = \sum_{n \in pares} x[n] W_N^{nk} + \sum_{n \in impares} x[n] W_N^{nk} =$$

$$= \sum_{r=0}^{N/2-1} x[2r] W_N^{2rk} + \sum_{r=0}^{N/2-1} x[2r+1] W_N^{(2r+1)k} =$$

$$= \sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{rk} + W_{N/2}^{r} \sum_{r=0}^{N/2-1} x[2r+1] W_N^{k} =$$

$$X[k] = F[k] + W_N^k G[k] \text{ para } k = \{0, ..., N/2-1\}$$

$F[k]$ y $G[k]$ son las DFT de N/2 puntos de $x[2r]$ (pares) y $x[2r+1]$ (impares).

FFT: Cómputo eficiente de la DFT

Ejemplo: Algoritmo en base 2 de diezmado temporal (hay algoritmos más rapidos)

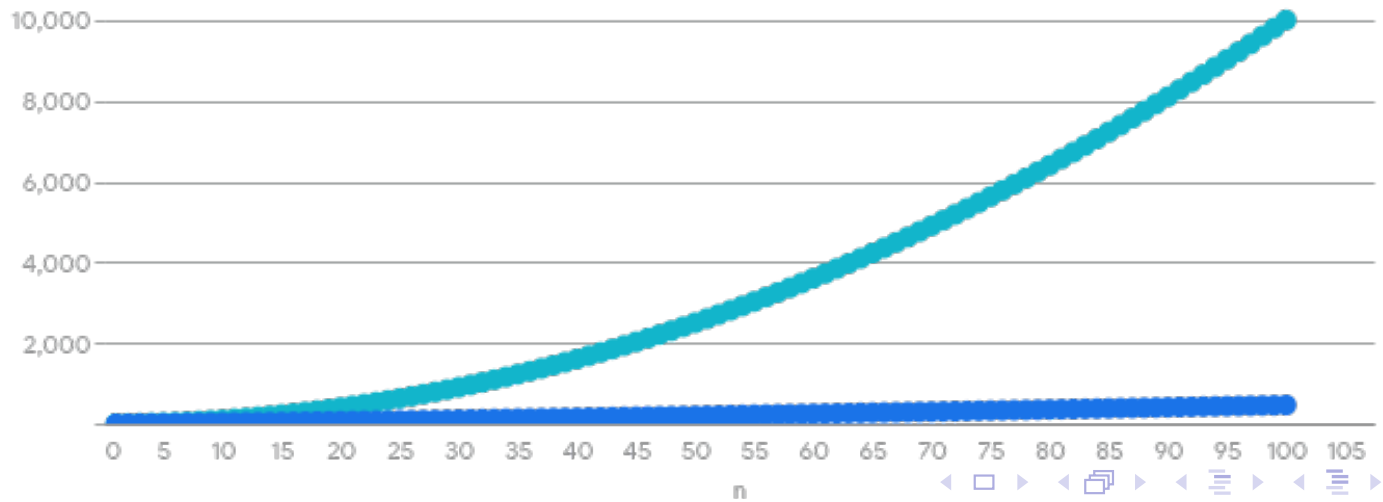
- $F[k]$ y $G[k]$ son periódicas $(N/2)$, podemos calcular el resto de valores:

$$X[k+N/2] = F[k] + W_N^{k+N/2} G[k] \text{ para } k = \{N/2, ..., N-1\}$$

- Finalmente:

$$X[k] = \begin{cases} X[k] = F[k] + W_N^k G[k] \text{ para } k = \{0, ..., N/2-1\} \\ X[k] = F[k] - W_N^k G[k] \text{ para } k = \{N/2, ..., N-1\} \end{cases}$$

- Se puede reducir la complejidad computacional de $O(N^2)$ a $O(N \cdot log(N))$
  - En el cálculo directo teníamos N^2 sumas + N^2 productos + otros.
  - Ahora tenemos $2 \cdot (\frac{N}{2})^2$ sumas + $2 \cdot (\frac{N}{2})^2$ productos + otros.
  - La división de la secuencia se puede repetir para calcular $F[k]$ y $G[k]$.



Cómputo eficiente de la respuesta del sistema FIR

- Respuesta de un sistema FIR: podemos usar el producto de DFTs.
  - Suponemos $x[n]$ de longitud L y $h[n]$ de longitud M.
  - Con L o M elevadas, el cálculo debe ser eficiente (FFT).
- Si $L >> M$ se puede simplificar el calculo.
  - Se divide $x[n]$ en subsecuencias de $L_D + M - 1$ valores, $L_D >> M$.
  - Método 1: solapamiento y almacenamiento.
  - Método 2: solapamiento y suma.

Representación de la DFT

- DFT: `X=fft(x)`, IDFT: `x=ifft(X)`
  - `x` es un vector con `N=length(x)` elementos,
  - `X` es su DFT con `N` elementos.
  - Para obtener más muestras de la DFT, rellenamos `x` con ceros.
  - Representamos el modulo y el argumento por separado.
  - Para representarlo frente a Ω discreta:
    - ★ `w=2*pi*[0:N-1]/N;`(rad/muestra)
    - ★ `f=[0:N-1]/N;`(1/muestra)
    - ★ `f=2*[0:N-1]/N;`(frec. normalizada)
  - Para representarlo frente a ω continua:
    - ★ `w=2*pi*[0:N-1]/N/Ts;`(rad/s)
    - ★ `f=[0:N-1]/N/Ts;`(Hz)

Resumen

- FFT - Diezmado temporal en base 2:
  - Divide sucesivamente en subsecuencias pares e impares.
  - La operación mariposa combina los resultados de las subsecuencias.

$$X[k] = \begin{cases} X[k] = F[k] + W_N^k G[k] \text{ para } k = \{0, ..., N/2 - 1\} \\ X[k] = F[k] - W_N^k G[k] \text{ para } k = \{N/2, ..., N - 1\} \end{cases}$$

  - Reduce la complejidad de $O(N^2)$ a $O(N \log_2 N)$.
- Representación:
  - Frente a Ω discreta:
    - ★ `w=2*pi*[0:N-1]/N;`(rad/muestra)
    - ★ `f=[0:N-1]/N;`(1/muestra)
    - ★ `f=2*[0:N-1]/N;`(frec. normalizada)
  - Frente a ω continua:
    - ★ `w=2*pi*[0:N-1]/N/Ts;`(rad/s)
    - ★ `f=[0:N-1]/N/Ts;`(Hz)